

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторная работа № 2 по курсу
«Операционные системы и архитектура компьютера»**

Выполнил: Федосеев Николай Владимирович

Группа: М8О-204БВ-25

Преподаватель: Н. А. Гапонов

Дата: 08.10.2026

Оценка: _____

Москва, 2026

Условие

Цель работы: создать дочерний процесс, запустить в нём отдельную программу и организовать передачу данных через канал.

Задание, вариант 1. Пользователь вводит команды вида: «число число число<newline>». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип `int`. Количество чисел может быть произвольным.

Метод решения

Родительская программа `parent` получает имя выходного файла, создаёт неименованный канал с помощью `pipe` и дочерний процесс с помощью `fork`. Родитель закрывает конец канала для чтения, а дочерний процесс - конец для записи. Через оставшиеся дескрипторы родитель передаёт данные, а дочерний процесс принимает их.

Дочерний процесс через `dup2` назначает конец канала своим стандартным вводом и вызывает `exec1`. Этот вызов заменяет программу в существующем процессе на `child`; новый процесс при этом не создаётся. Имя выходного файла передаётся в `argv[1]`. Дескриптор стандартного ввода сохраняется при успешном запуске новой программы.

Родитель читает команды через `getline` и передаёт их в канал функцией `write_all`. Канал передаёт поток байтов, а не отдельные сообщения. Конец строки служит границей команды. В дочерней программе `getline` собирает строки из этого потока, `parse_sum` разбирает числа, затем сумма записывается в файл.

После EOF родитель закрывает конец канала для записи. Ребёнок дочитывает оставшиеся данные и получает EOF, когда в канале больше нет данных и все дескрипторы записи закрыты. Родитель ждёт завершения ребёнка через `waitpid` и проверяет его статус.

Описание программы

Структура файлов

Таблица 1 – Основные файлы проекта

Файл	Назначение
<code>src/parent.c</code>	Ввод, создание канала и процесса, запуск ребёнка, передача строк, ожидание результата.
<code>src/child.c</code>	Разбор чисел, проверка суммы, запись выходного файла.
<code>src/CMakeLists.txt</code>	Сборка двух программ в стандарте C17.

Основные типы данных и функции

`int pipe_fd[2]` хранит дескрипторы канала: индекс 0 соответствует чтению, индекс 1 записи. `pid_t child_pid` хранит PID дочернего процесса. `ssize_t` используется для результата `getline` и `write`: успешный результат неотрицателен, ошибка обозначается значением `-1`. Длина и вместимость буфера хранятся в `size_t`.

Функция `write_all(int fd, const char *data, size_t size)` есть в обеих программах. Она записывает весь буфер, продолжает запись после частичного успеха и повторяет вызов при `EINTR`. Возвращает 0 при успехе и `-1` при ошибке.

`parse_sum(const char *line, int *sum_out)` последовательно разбирает десятичные числа через `strtol`. Возвращает 0 при успехе, -1 при некорректной или пустой строке, -2 при переполнении суммы. Результат записывается по указателю только после успешного разбора всей строки. Перед `strtol` сбрасывается `errno`; проверяются отсутствие числа, `ERANGE`, диапазон `int` и разделитель после числа.

`getline` выделяет и увеличивает буфер по мере необходимости. Фиксированного ограничения на число элементов в строке в программе нет. Буферы освобождаются через `free`.

Системные вызовы

Таблица 2 – Операции с процессами, каналом и файлом

Функция в коде	Назначение
<code>pipe</code>	Создаёт канал
<code>fork</code>	Создаёт ребёнка
<code>dup2</code>	Переназначает дескриптор 0 на конец канала для чтения
<code>execl</code>	Запускает <code>child</code> в текущем процессе через <code>execve</code>
<code>open</code>	Создаёт или очищает выходной файл
<code>write</code>	Передаёт команды в канал и записывает суммы в файл
<code>close</code>	Закрывает ненужные концы канала и выходной файл
<code>waitpid</code>	Ждёт конкретного ребёнка и получает его статус
<code>getpid, getppid</code>	Получают PID для диагностического вывода
<code>signal</code>	Устанавливает игнорирование <code>SIGPIPE</code> , чтобы при закрытии канала дочерним процессом родитель мог обработать ошибку записи <code>EPIPE</code>
<code>_exit</code>	Завершает ребёнка при ошибке до успешного <code>exec</code>

`getline` и `fprintf` не являются системными вызовами, но в своей реализации `libc` вызывают `read` и `write`.

Выходной файл открывается с флагами `O_WRONLY | O_CREAT | O_TRUNC`. Если он существовал, старое содержимое удаляется. Для нового файла запрашиваются права 0644. Программа не создаёт недостающие каталоги.

Ключевые фрагменты кода

После создания канала и `fork` ребёнок закрывает конец записи и перенаправляет стандартный ввод. Проверка равенства дескрипторов нужна, чтобы не закрыть дескриптор 0, если конец чтения уже оказался стандартным вводом.

Листинг 1 – Перенаправление ввода и запуск `child` (`parent.c`)

```

if (pipe_fd[0] != STDIN_FILENO) {
    if (dup2(pipe_fd[0], STDIN_FILENO) == -1) {
        perror("dup2");
        _exit(EXIT_FAILURE);
    }
    if (close(pipe_fd[0]) == -1) {
        perror("close read end");
    }
}

```

```

        _exit(EXIT_FAILURE);
    }
}

execl(CHILD_PATH, "child", filename, (char *) NULL);
perror("execl");
_exit(EXIT_FAILURE);

```

При успешном `execl` управление в этот код не возвращается. `perror` и `_exit` выполняются только при ошибке. В отличие от `exit`, `_exit` не сбрасывает унаследованные буферы `stdio`.

Один `write` может записать только часть буфера. Следующая итерация передаёт оставшуюся часть; `EINTR` означает, что вызов нужно повторить.

Листинг 2 – Полная запись буфера (`parent.c`, `child.c`)

```

while (written < size) {
    ssize_t res = write(fd, data + written, size - written);
    if (res > 0) {
        written += (size_t) res;
    } else if (res == 0) {
        errno = EIO;
        return -1;
    } else if (errno != EINTR) {
        return -1;
    }
}

```

Перед сложением проверяется, поместится ли результат в `int`. Само переполняющее сложение не выполняется.

Листинг 3 – Проверка числа и суммы (`child.c`)

```

if (current == end || errno == ERANGE ||
    value < INT_MIN || value > INT_MAX) return -1;
if (*end != '\0' && !isspace((unsigned char)*end)) return -1;

if ((value > 0 && sum > INT_MAX - value) ||
    (value < 0 && sum < INT_MIN - value)) return -2;

sum += (int) value;

```

После передачи команд родитель закрывает конец записи и ждёт ребёнка. Прерывание ожидания сигналом не считается завершением процесса, поэтому при `EINTR` ожидание повторяется.

Листинг 4 – Ожидание дочернего процесса (parent.c)

```
do {  
    wait_result = waitpid(child_pid, &status, 0);  
} while (wait_result == -1 && errno == EINTR);
```

Далее WIFEXITED проверяет обычное завершение, а WEXITSTATUS извлекает код. WIFSIGNALED и WTERMSIG используются, если процесс завершён сигналом. Ненулевой код ребёнка или ошибка родителя приводит к EXIT_FAILURE у родителя.

Обработка ошибок

Имя файла не должно быть пустым или содержать встроенный NUL. Такой же NUL проверяется в каждой команде до разбора чисел. Допустимы знаки чисел, пробелы и табуляция; дробные числа, буквы, слитные токены и пустые команды отклоняются.

При ошибке дочерняя программа сообщает номер строки и прекращает обработку. Ранее записанные суммы остаются в файле. Родитель игнорирует SIGPIPE: если ребёнок закрыл канал, запись возвращает ошибку EPIPE, которую родитель может обработать перед waitpid.

Сборка и запуск

Используется CMake не ниже 3.21. Включены -Wall -Wextra -Wpedantic. Макрос _POSIX_C_SOURCE=200809L открывает объявления POSIX-функций, включая getline, при строгом режиме C17.

```
cmake -S src -B build -DCMAKE_BUILD_TYPE=Debug  
cmake --build build --parallel  
./build/parent
```

CMake передаёт абсолютный путь к собранному child через макрос CHILD_PATH.

При запуске сначала вводится имя файла, затем команды. После последней строки нужно нажать Enter и Ctrl+D на пустой строке.

Демонстрация работы strace

Программа запущена из корня проекта командой:

```
mkdir -p report/inc  
strace -f -yy -s 256 -o report/inc/strace.txt ./build/parent
```

Флаг -f включает отслеживание дочернего процесса, -уу подписывает дескрипторы, -s 256 увеличивает лимит отображаемых символов, -o сохраняет вывод strace в указанный файл.

Ввод при запуске:

```
res.txt  
1 2 3  
3 4  
5
```

После последней строки ввод завершён через Ctrl+D. PID родителя — 110201, PID ребёнка — 110204. Полный вывод strace приведён в приложении Б. Номера строк ниже соответствуют этому файлу.

В начале вывода видна загрузка библиотек. Основная работа программы начинается с чтения имени файла на строке 33. Пометки `unfinished` и `resumed` обозначают начало вызова и продолжение его записи в выводе strace. Части одного вызова сопоставляются по PID и имени вызова.

Создание канала и дочернего процесса

Строки 34 и 36:

```
110201 pipe2([3<pipe:[4014953]>, 4<pipe:[4014953]>], 0) = 0
110201 clone(child_stack=NULL,
↳ flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
↳ child_tidptr=0x7f9d26825a10) = 110204
```

`pipe2` создаёт канал с дескрипторами 3 для чтения и 4 для записи. Подпись `pipe:[4014953]` обозначает один и тот же канал. В коде используется `fork`, а в выводе strace виден `clone`; родитель получает PID созданного ребёнка 110204.

После `fork` родитель закрывает свой конец чтения, а ребёнок свой конец записи. Строки 42–45:

```
110201 close(3<pipe:[4014953]> <unfinished ...>
110204 close(4<pipe:[4014953]> <unfinished ...>
110201 <... close resumed>) = 0
110204 <... close resumed>) = 0
```

У каждого процесса своя таблица дескрипторов. Поэтому закрытие дескриптора 3 у родителя не закрывает конец чтения у ребёнка.

Перенаправление ввода и запуск child

Родитель устанавливает игнорирование SIGPIPE. В строках 46 и 48 это видно как `rt_sigaction` с `sa_handler=SIG_IGN`. В данном запуске ошибок записи в канал не было.

Ребёнок перенаправляет стандартный ввод на канал и закрывает лишний дескриптор. Строки 47, 49, 51 и 53:

```
110204 dup2(3<pipe:[4014953]>, 0</dev/pts/1<char 136:1>> <unfinished ...>
110204 <... dup2 resumed>) = 0<pipe:[4014953]>
110204 close(3<pipe:[4014953]> <unfinished ...>
110204 <... close resumed>) = 0
```

После `dup2` дескриптор 0 ребёнка связан с каналом. Дескриптор 3 закрывается, но дескриптор 0 остаётся открытым.

Строки 54 и 57:

```
110204 execve("/home/nikallow/Code/1-mai/3-os/laba2/build/child", ["child",
↳ "res.txt"], 0x7fff1911f118 /* 87 vars */ <unfinished ...>
110204 <... execve resumed>) = 0
```

Вызов `exec1` из кода выполняется через `execve`. В процессе 110204 запускается `child`; PID не меняется. Имя файла `res.txt` передано через `argv[1]`, стандартный ввод остаётся связан с каналом.

Открытие файла и передача команд

Строка 86:

```
110204 openat(AT_FDCWD</home/nikallow/Code/1-mai/3-os/laba2>, "res.txt",
↳ O_WRONLY|O_CREAT|O_TRUNC, 0644) =
↳ 3</home/nikallow/Code/1-mai/3-os/laba2/res.txt>
```

Ребёнок открывает `res.txt` для записи с флагами создания и очистки файла. `open` из кода отображается в выводе `strace` как `openat`. Полученный дескриптор 3 теперь относится к выходному файлу.

Первая команда прослеживается в строках 94–98:

```
110201 <... read resumed>, "1 2 3\n", 1024) = 6
110201 write(4<pipe:[4014953]>, "1 2 3\n", 6) = 6
110204 <... read resumed>, "1 2 3\n", 4096) = 6
110201 read(0</dev/pts/1<char 136:1>> <unfinished ...>
110204 write(3</home/nikallow/Code/1-mai/3-os/laba2/res.txt>, "6\n", 2) = 2
```

На строке 94 родитель получает из терминала 6 байтов: 1 2 3 и перевод строки. На строке 95 он записывает эти байты в канал, на строке 96 ребёнок их читает. Вызов чтения ребёнка начался на строке 93. На строке 97 родитель уже ждёт следующую команду, а на строке 98 ребёнок записывает сумму 6 и перевод строки в файл.

`getline` виден в выводе `strace` через вызовы `read`. Разбор чисел и сложение выполняются внутри процесса, поэтому отдельных системных вызовов для них нет. Для команд 3 4 и 5 ребёнок записывает суммы 7 и 5 на строках 104 и 110. В результате файл содержит:

```
6
7
5
```

EOF и завершение процессов

Строки 112–119:

```
110201 <... read resumed>, "", 1024) = 0
110201 close(4<pipe:[4014953]>) = 0
110204 <... read resumed>, "", 4096) = 0
110201 wait4(110204 <unfinished ...>
110204 close(3</home/nikallow/Code/1-mai/3-os/laba2/res.txt>) = 0
110204 exit_group(0) = ?
110204 +++ exited with 0 +++
110201 <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) =
↳ 110204
```

На строке 112 родитель получает EOF из терминала: `read` возвращает 0. На строке 113 он закрывает конец канала для записи. Ребёнок ранее закрыл свою копию этого конца, поэтому открытых дескрипторов записи больше нет. После чтения всех данных ребёнок тоже получает EOF: на строке 114 его `read` возвращает 0.

Родитель ждёт ребёнка через `waitpid`, который отображается как `wait4`. Ребёнок закрывает файл и завершает работу с кодом 0. На строке 119 родитель получает его статус: `WIFEXITED(s)` подтверждает обычное завершение, `WEXITSTATUS(s) == 0` подтверждает успешный код возврата.

На строке 120 родителю доставляется `SIGCHLD`. Затем он выводит статус ребёнка и также завершается с кодом 0, что видно на строках 121–123.

Выводы

Сделаны две программы, которые передают строки через pipe и записывают суммы в файл. В решении использованы создание процесса, замена его программы, перенаправление стандартного ввода и ожидание завершения ребёнка. Обработаны ошибки системных вызовов, некорректных числа и переполнения суммы. Закрытие лишних концов канала позволяет передать EOF, а waitpid позволяет получить статус дочернего процесса.

Источники

1. Linux man-pages: pipe(7)
2. Linux man-pages: fork(2)
3. Linux man-pages: dup(2), dup2(2)
4. Linux man-pages: exec(3), execl(3)
5. Linux man-pages: wait(2), waitpid(2)
6. Linux man-pages: write(2)

Приложение А. Исходный код программы и сборка

Листинг А.1 – Файл src/parent.c

```
#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

static int write_all(int fd, const char *data, size_t size) {
    size_t written = 0;

    while (written < size) {
        ssize_t res = write(fd, data + written, size - written);

        if (res > 0) {
            written += (size_t) res;
        } else if (res == 0) {
            errno = EIO;
            return -1;
        } else if (errno != EINTR) {
            return -1;
        }
    }

    return 0;
}

int main(void) {
    char *filename = NULL;
    size_t capacity = 0;
    ssize_t length = getline(&filename, &capacity, stdin);

    if (length == -1) {
        if (ferror(stdin)) perror("getline filename");
        else fprintf(stderr, "Missing output filename\n");
        free(filename);
        return EXIT_FAILURE;
    }

    if (length > 0 && filename[length - 1] == '\n') filename[--length] = '\0';
    if (length > 0 && filename[length - 1] == '\r') filename[--length] = '\0';

    if (length == 0 || memchr(filename, '\0', (size_t) length) != NULL) {
        fprintf(stderr, "Invalid output filename\n");
        free(filename);
        return EXIT_FAILURE;
    }
}
```

```

int pipe_fd[2];
if (pipe(pipe_fd) == -1) {
    perror("pipe");
    free(filename);
    return EXIT_FAILURE;
}

pid_t child_pid = fork();
if (child_pid == -1) {
    perror("fork");
    if (close(pipe_fd[0]) == -1) perror("close read end");
    if (close(pipe_fd[1]) == -1) perror("close write end");
    free(filename);
    return EXIT_FAILURE;
}

if (child_pid == 0) {
    if (close(pipe_fd[1]) == -1) {
        perror("close write end");
        _exit(EXIT_FAILURE);
    }

    if (pipe_fd[0] != STDIN_FILENO) {
        if (dup2(pipe_fd[0], STDIN_FILENO) == -1) {
            perror("dup2");
            _exit(EXIT_FAILURE);
        }
        if (close(pipe_fd[0]) == -1) {
            perror("close read end");
            _exit(EXIT_FAILURE);
        }
    }

    execl(CHILD_PATH, "child", filename, (char *) NULL);
    perror("execl");
    _exit(EXIT_FAILURE);
}

free(filename);
int failed = 0;

if (close(pipe_fd[0]) == -1) {
    perror("close read end");
    failed = 1;
}

if (!failed && signal(SIGPIPE, SIG_IGN) == SIG_ERR) {
    perror("signal SIGPIPE");
    failed = 1;
}

```

```

fprintf(stderr, "parent PID: %ld, child PID: %ld\n", (long) getpid(),
↳ (long) child_pid);

char *line = NULL;
size_t line_capacity = 0;

while (!failed) {
    ssize_t line_length = getline(&line, &line_capacity, stdin);
    if (line_length == -1) {
        if (ferror(stdin)) {
            perror("getline command");
            failed = 1;
        }
        break;
    }

    if (write_all(pipe_fd[1], line, (size_t) line_length) == -1) {
        perror("write pipe");
        failed = 1;
        break;
    }
}

free(line);

if (close(pipe_fd[1]) == -1) {
    perror("close write end");
    failed = 1;
}

int status;
pid_t wait_result;
do {
    wait_result = waitpid(child_pid, &status, 0);
} while (wait_result == -1 && errno == EINTR);

if (wait_result == -1) {
    perror("waitpid");
    return EXIT_FAILURE;
}

if (WIFEXITED(status)) {
    fprintf(stderr, "child exited with status: %d\n",
↳ WEXITSTATUS(status));
    if (WEXITSTATUS(status) != EXIT_SUCCESS) failed = 1;
} else if (WIFSIGNALED(status)) {
    fprintf(stderr, "child killed by signal: %d\n", WTERMSIG(status));
    failed = 1;
} else {
    fprintf(stderr, "Unexpected child status\n");
    failed = 1;
}

```

```

    return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}

```

Листинг А.2 – Файл src/child.c

```

#include <ctype.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>
#include <limits.h>
#include <string.h>

static int write_all(int fd, const char *data, size_t size) {
    size_t written = 0;

    while (written < size) {
        ssize_t res = write(fd, data + written, size - written);

        if (res > 0) {
            written += (size_t) res;
        } else if (res == 0) {
            errno = EIO;
            return -1;
        } else if (errno != EINTR) {
            return -1;
        }
    }

    return 0;
}

static int parse_sum(const char *line, int *sum_out) {
    const char *current = line;
    int sum = 0;
    int found = 0;

    while (*current != '\0') {
        while (isspace((unsigned char)*current)) current++;
        if (*current == '\0') break;

        errno = 0;
        char *end;
        long value = strtol(current, &end, 10);

        if (current == end || errno == ERANGE || value < INT_MIN || value >
            ↪ INT_MAX) return -1;
        if (*end != '\0' && !isspace((unsigned char)*end)) return -1;
    }
}

```

```

        if ((value > 0 && sum > INT_MAX - value) || (value < 0 && sum <
            ↪ INT_MIN - value)) return -2;

        sum += (int) value;
        found = 1;
        current = end;
    }

    if (!found) return -1;
    *sum_out = sum;
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: child <output-file>\n");
        return EXIT_FAILURE;
    }

    int output_fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    ↪ //rw-r--r--
    if (output_fd == -1) {
        perror("open output file");
        return EXIT_FAILURE;
    }

    fprintf(stderr, "child PID: %ld, parent PID: %ld\n", (long) getpid(),
        ↪ (long) getppid());

    char *line = NULL;
    size_t capacity = 0;
    size_t line_number = 0;
    int failed = 0;

    while (1) {
        ssize_t length = getline(&line, &capacity, stdin);
        if (length == -1) {
            if (ferror(stdin)) {
                perror("getline stdin");
                failed = 1;
            }
            break;
        }

        line_number++;
        // line is invalid if '\0' present in mid of it
        if (memchr(line, '\0', (size_t) length) != NULL) {
            fprintf(stderr, "Invalid command on line %zu: NUL byte\n",
                ↪ line_number);
            failed = 1;
            break;
        }
    }
}

```

```

int sum;
int parse_result = parse_sum(line, &sum);
if (parse_result != 0) {
    if (parse_result == -2) {
        fprintf(stderr, "Integer sum overflow on line %zu\n",
            ↪ line_number);
    } else {
        fprintf(stderr, "Invalid command on line %zu\n", line_number);
    }
    failed = 1;
    break;
}

char output[32];
int output_length = snprintf(output, sizeof(output), "%d\n", sum);
if (output_length < 0 || (size_t) output_length >= sizeof(output)) {
    fprintf(stderr, "Failed to format sum\n");
    failed = 1;
    break;
}

if (write_all(output_fd, output, (size_t) output_length) == -1) {
    perror("write output file");
    failed = 1;
    break;
}
}

free(line);

if (close(output_fd) == -1) {
    perror("close output file");
    failed = 1;
}

return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}

```

Листинг A.3 – Файл src/CMakeLists.txt

```

cmake_minimum_required(VERSION 3.21)

project(lab2 C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

add_compile_definitions(_POSIX_C_SOURCE=200809L)

add_executable(parent parent.c)

```

```

add_executable(child child.c)

target_compile_options(parent PRIVATE -Wall -Wextra -Wpedantic)
target_compile_options(child PRIVATE -Wall -Wextra -Wpedantic)

add_dependencies(parent child)
target_compile_definitions(parent PRIVATE CHILD_PATH="$<TARGET_FILE:child>")

```

Листинг А.4 – Файл report/Dockerfile

```

FROM texlive/texlive:latest

WORKDIR /work/report

CMD ["sh", "-c", "pdflatex -interaction=nonstopmode -halt-on-error report.tex
↪  && pdflatex -interaction=nonstopmode -halt-on-error report.tex"]

```

Листинг А.5 – Файл report/Makefile

```

MAIN    = report
LATEX ?= pdflatex
FLAGS   = -interaction=nonstopmode -halt-on-error
SOURCES = $(MAIN).tex preamble.tex titlepage.tex $(wildcard inc/*) \
           $(wildcard ../src/*.c) ../src/CMakeLists.txt Dockerfile Makefile

$(MAIN).pdf: $(SOURCES)
              $(LATEX) $(FLAGS) $(MAIN).tex
              $(LATEX) $(FLAGS) $(MAIN).tex

xelatex:
          $(MAKE) -B LATEX=xelatex $(MAIN).pdf

clean:
          rm -f *.aux *.log *.out *.lol *.toc *.fls *.fdb_latexmk *.synctex.gz

.PHONY: xelatex clean

```

Листинг Б.1 – Трассировка parent и child

16


```

91 110204 brk(0x19eba000) = 0x19eba000
92 110204 fstat(0<pipe:[4014953]>, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
93 110204 read(0<pipe:[4014953]> <unfinished ...>
94 110201 <... read resumed>, "1 2 3\n", 1024) = 6
95 110201 write(4<pipe:[4014953]>, "1 2 3\n", 6) = 6
96 110204 <... read resumed>, "1 2 3\n", 4096) = 6
97 110201 read(0</dev/pts/1<char 136:1>> <unfinished ...>
98 110204 write(3</home/nikallow/Code/1-mai/3-os/laba2/res.txt>, "6\n", 2) = 2
99 110204 read(0<pipe:[4014953]> <unfinished ...>
100 110201 <... read resumed>, "3 4\n", 1024) = 4
101 110201 write(4<pipe:[4014953]>, "3 4\n", 4) = 4
102 110204 <... read resumed>, "3 4\n", 4096) = 4
103 110201 read(0</dev/pts/1<char 136:1>> <unfinished ...>
104 110204 write(3</home/nikallow/Code/1-mai/3-os/laba2/res.txt>, "7\n", 2) = 2
105 110204 read(0<pipe:[4014953]> <unfinished ...>
106 110201 <... read resumed>, "5\n", 1024) = 2
107 110201 write(4<pipe:[4014953]>, "5\n", 2) = 2
108 110204 <... read resumed>, "5\n", 4096) = 2
109 110201 read(0</dev/pts/1<char 136:1>> <unfinished ...>
110 110204 write(3</home/nikallow/Code/1-mai/3-os/laba2/res.txt>, "5\n", 2) = 2
111 110204 read(0<pipe:[4014953]> <unfinished ...>
112 110201 <... read resumed>, "", 1024) = 0
113 110201 close(4<pipe:[4014953]>) = 0
114 110204 <... read resumed>, "", 4096) = 0
115 110201 wait4(110204 <unfinished ...>
116 110204 close(3</home/nikallow/Code/1-mai/3-os/laba2/res.txt>) = 0
117 110204 exit_group(0) = ?
118 110204 +++ exited with 0 +++
119 110201 <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 110204
120 110201 --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=110204, si_uid=1000, si_status=0,
    ↪ si_utime=0, si_stime=0} ---
121 110201 write(2</dev/pts/1<char 136:1>>, "child exited with status: 0\n", 28) = 28
122 110201 exit_group(0) = ?
123 110201 +++ exited with 0 +++

```